Beyond Web5: A Substrate Map for Operator-Resident Agentic Systems

Zachary Holwerda
Airlock Labs
Detroit, MI
zac@airlocklabs.io

Abstract

The progression Web1 → Web2 → Web3 → Web4 is often summarized as read → read-write → own → inhabit. Web5 has been positioned variously as decentralized identity, as personal-server federation, or as a vague "next layer," but there is little consensus on what a concrete operator-resident substrate beyond Web4 should contain. We describe and instrument a working substrate that orchestrates the prior four layers into a single operator-resident shell. We identify seven primitives — phone-anchored decentralized identifiers (Skeet), bot-federated identity in a biometric-gated local shell (Spacebar), signed bivector closures for event logging (BCODE), behavioral-fidelity scoring (DECF + a 17-persona dataset, ConstellationBench), agent inheritance patterns (OctoAgent), token-economy coordination ($MOTION with ACODE-free / BCODE-paid-or-earned separation), and matrix-style federation transport (Matrix / bitchat) — and show how they compose into a running system with a fully tested state service (4/4 canonical "wires" green, 25/26 behavioral tests passing). ConstellationBench serves as an empirical lens: we measure behavioral fidelity of frontier models across personas and use the results to stress-test inheritance and logging primitives. We release the dataset, runtime, and core substrate components as open artifacts (ACODE-free), enabling other operator-resident systems to reuse individual primitives or benchmark alternative substrate designs against the same behavioral lens.

1 Introduction

The last decade of web and AI infrastructure has produced a familiar ladder: Web1 (read — static pages, one-way information flow), Web2 (read + write — social platforms, user-generated content), and Web3 (own — tokenized assets, on-chain identifiers, decentralized wallets) [Berners-Lee, 1989, O'Connor, 2023]. Recent proposals add two further layers: Web4 (filter / express — substrate-level routing, provenance, and expression governance) and Web5 (inhabit / identify — operator-resident identity and control surfaces) [Block Inc. (TBD), 2022, Sambra et al., 2016, Mühle et al., 2018], but largely stop at architectural slogans rather than concrete, measurable systems. In this work we take a complementary approach: instead of debating the exact definition of Web4 and Web5, we implement and instrument an operator-resident substrate that *realizes* both verbs explicitly — a translation-layer provenance filter (Web4) and a biometric-gated local identity shell (Web5) — treating Web1–3 as input surfaces. The resulting system combines seven primitives — identity, logging, behavioral scoring, inheritance, incentives, and federation — into a working "operator OS" whose behavior we can probe empirically using a 17-persona benchmark (ConstellationBench) and a fully wired state service with 4/4 health, loading, persistence, and warm-start checks passing.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1.1 From Web5 slogans to operator-resident substrates

Most Web5 discussions to date focus on protocol proposals: decentralized identifiers (DIDs) and verifiable credentials [W3C, 2022], personal data stores and Solid pods [Sambra et al., 2016], or high-level visions of "the web of identity" [Block Inc. (TBD), 2022]. These lines of work are important, but they leave open a concrete systems question: *what primitives are actually required to operate a real, operator-resident substrate that sits above existing web layers and interacts with them in practice?*

We address this question empirically. Rather than specifying a new protocol and leaving its deployment as future work, we start from a running system used for day-to-day development, then factor out the primitives that proved necessary to make it observable, auditable, and controllable by a human operator. Concretely, our implementation exposes seven reusable primitives:

1. phone-anchored decentralized identifiers (Skeet);
2. a biometric-gated local control shell that federates identities (Spacebar v0);
3. signed bivector closures for event logging (BCODE);
4. behavioral-fidelity scoring via DECF and a 17-persona dataset (ConstellationBench);
5. agent inheritance patterns for specialized assistants (OctoAgent);
6. a token-economy layer for incentives ($MOTION with ACODE/BCODE split);
7. matrix-style federation transport (Matrix / bitchat).

Each primitive is implemented in a real codebase, exported as an open artifact, and evaluated in the context of the running substrate.

1.2 ConstellationBench as an empirical lens

ConstellationBench is a 17-persona benchmark designed to measure behavioral fidelity of large language models along multiple DECF axes: directionality, explicitness, coherence, and fidelity [Holwerda, 2026d]. Rather than being the sole focus of this paper, ConstellationBench plays the role of an *empirical lens* on the substrate: we use it to probe how different primitives behave when populated with real personas and real models.

First, we use DECF scores across personas to quantify how consistently frontier models reproduce intended behavioral profiles, providing a concrete notion of "behavioral fidelity" that goes beyond aggregate task accuracy. Second, we instrument the substrate so that every evaluation run produces signed BCODE closures and serialized context, allowing us to study how inheritance patterns (OctoAgent) and logging interact in practice. The Phase-0 wiring — a state service, persona loader, session persistence, and warm-start behavior, with 4/4 canonical wires green and 25/26 tests passing — serves as our reproducibility receipt: it specifies exactly which components must be operational for other groups to replicate our experiments or to plug in alternative primitives.

1.3 Contributions

Our contributions are threefold:

- **Substrate map.** We identify seven primitives that, in combination, are sufficient to operate a working operator-resident substrate above existing web layers, and we provide an architectural map showing how they compose into a single local control shell.
- **Empirical lens.** We use ConstellationBench as a behavioral-fidelity lens on this substrate, reporting results for multiple frontier models across 17 personas and analyzing how inheritance and logging primitives behave under stress.
- **Open artifacts.** We release the ConstellationBench dataset, evaluation runtime, and core substrate components (ACODE-free) so that other operator-resident systems can reuse individual primitives or benchmark alternative designs against the same empirical lens.

2 The Substrate Map

Rather than proposing yet another abstract "Web5" protocol, we start from a running system and factor out the primitives that proved necessary to operate it as an operator-resident substrate. Concretely, our

implementation exposes seven reusable primitives that span identity, logging, behavioral evaluation, agent inheritance, incentives, and transport. Table 1 summarizes these primitives, their roles in the substrate, and the canonical implementations we release; subsequent sections show how they compose into a single local control shell and how their behavior can be probed empirically via ConstellationBench.

2.0.1 Table 1: The Seven Substrate Primitives

#	Primitive	Web layer	Role in substrate	Canonical implementation
1	Phone-anchored DIDs (Skeet)	Web3 + Web5	DID-backed identity layer that can be bound to phone numbers, social accounts, or eSIMs; underlies Free / Pro / Enterprise tiers	Skeet bot bindings + carrier eSIM bindings
2	Spacebar (operator-resident shell)	Web5 (inhabit / identify)	Biometric-gated local cockpit that federates inbound identity events and agent interactions into a single operator surface	RN+Expo native client; humbble fork base
3	BCODE (signed bivector closures)	Web4 (filter / express)	Cryptographically signed event log; records agent actions and substrate state transitions as bivectors with side-channel-safe serialization	`bcode/closure.py`
4	DECF + ConstellationBench	Web4	Behavioral-fidelity evaluation: DECF scores + 17-persona dataset provide an empirical lens for frontier-model behavior across personas	constellation-bench-hf dataset + scripts
5	OctoAgent (agent inheritance)	Web4 + Web5	Encodes how specialized agents inherit parameters, memories, and behavior from a base operator profile while remaining auditable	OctoAgent inheritance protocol

#	Primitive	Web layer	Role in substrate	Canonical implementation
6	$MOTION + ACODE/BCODE split	Web3	Incentive layer separating free substrate code (ACODE) from paid-or-earned behavior (BCODE) and tying it to a token economy	$MOTION on Sonr-chain + ACODE-free monetization model
7	Matrix/bitchat federation	Web2 + Web5	Transport mesh for messages and events (BEAM RELAY), enabling substrate-resident agents to interoperate across services	Matrix protocol + bitchat mesh

In §3 we use ConstellationBench (primitive 4) as an empirical lens on the substrate, with particular attention to how primitives 3 (BCODE) and 5 (OctoAgent) behave under stress. In §4 we provide the Phase-0 reproducibility receipt covering primitives 1, 2, and 7. §5 addresses side-channel safety and differentiation against the gatekept-enterprise pattern (HiddenLayer) for the Web4 compute slot. §6 maps the seven primitives to a tier ladder (Free / Pro / Enterprise) that operators traverse as they grow from social-account auth to carrier-attested identity. §7 discusses PACT — a protocol-layer contribution emerging from the substrate's identity-aware-authorization needs — and future verticals.

3 Empirical motivation: what ConstellationBench actually shows

The substrate map in §2 is architectural; this section explains why those particular pieces and abstractions are necessary at all. ConstellationBench provides the empirical lens: instead of scoring models only on generic task success, it measures two orthogonal dimensions that matter for an operator-resident system — did the model solve the problem, and did it behave like the specific persona it claimed to be. Using that lens across a variety of models and orchestration patterns yields three consistent, somewhat uncomfortable findings: (i) "budget" models can beat frontier models on persona fidelity, (ii) a non-speaking audience agent can materially improve quality and fidelity at zero token cost, and (iii) multi-agent teams beyond a small size tend to collapse toward generic behavior rather than diversifying. These results are what drive the substrate toward explicit persona metrics, audience-aware orchestration, and a protocol layer that treats identity and trace as first-class objects rather than comments in a prompt.

3.1 Setup and evaluation lens

Experiments use ConstellationBench, a persona-sensitive benchmark that scores model outputs along two orthogonal axes: task quality (did the answer solve the problem?) and persona fidelity (did it sound and behave like the specified agent, not like a generic assistant). Models are evaluated under the same prompt scaffolds, tools, and scoring rubric to keep comparisons reproducible rather than anecdotal. Each condition is run across a panel of personalities so that results reflect structural effects, not idiosyncrasies of one archetype [Holwerda, 2026d].

3.2 Headline result: budget models beat frontier on persona fidelity

The first result is counter-intuitive if one only tracks leaderboard scores: smaller "budget" models (Anthropic Haiku and Sonnet) outperform frontier-class models (Opus, GPT-4-class) on persona fidelity by roughly 8 percentage points while achieving comparable task quality. In other words,

cheaper models stay "in character" more reliably, even though larger models score higher on generic benchmarks.

Two features make this result publication-grade rather than a one-off anomaly:

- The effect is consistent across multiple personas and tasks, not driven by a single outlier character.
- The experiment is inexpensive to reproduce: the headline budget-vs-frontier subset costs on the order of $60 of API spend (§8.2 details a $115 full protocol covering all 22 models × 17 personas × 7 benchmarks); other groups can verify or refute either tier without bespoke infrastructure.

The practical implication is that fidelity is not a simple monotone function of model scale. Once a model has been heavily trained and reinforced to act like a safe, polite, generic assistant, adding more parameters and training steps appears to strengthen that generic voice faster than it strengthens the capacity to inhabit distinct personas. The substrate therefore cannot assume "bigger is better" for agentic work; it must be able to route to cheaper models when fidelity, not raw capability, is the primary objective.

3.3 The alignment trap: generic safety vs specific identity

The fidelity gap points to a broader pattern the experiments keep surfacing: what can be called an "alignment trap." Modern frontier models are aligned, in the safety sense, to be inoffensive, broadly helpful, and stylistically uniform. That is appropriate for single-assistant experiences, but it comes at a cost for multi-agent or persona-specific systems.

Empirically, as alignment pressure and generic safety training increase, outputs converge toward a narrow band of "default assistant" behavior even when the prompt explicitly specifies a different character. In ConstellationBench runs, this shows up as:

- High task scores but middling persona scores for frontier models under strong safety scaffolds.
- Higher persona scores for smaller models or less-aligned checkpoints under identical persona instructions.

The alignment trap, then, is the regime in which marginal safety training continues to improve generic assistant behavior but actively erodes the system's ability to realize distinct, inspectable personas. A substrate that cares about behavioral fidelity must treat alignment as a multi-objective problem: keep safety constraints, but do not allow them to collapse the personality space to a single point.

3.4 Audience effect: the "silent Guardian" as stabilizer

A second robust effect is the "audience" or "silent Guardian" phenomenon. Adding an additional, non-generating agent to the system prompt — a named reviewer or overseer that never speaks to the user but is described as evaluating the assistant's work — produces a measurable improvement in output quality and persona fidelity, despite consuming no extra tokens at inference time.

Across runs, the presence of a silent Guardian agent yields approximately:

- +14% relative lift in task quality.
- +10.7% relative lift in persona fidelity.

Mechanistically, the effect looks less like classical self-critique and more like social priming. The base model is prompted to treat its own output as something that will be watched and graded by a peer or supervisor, and its behavior shifts accordingly. The key is that the Guardian never generates text or calls tools; it exists only as a parametric audience in the system prompt.

For the substrate, this suggests that some of the benefits usually attributed to more complex multi-agent architectures can be achieved by simpler orchestration patterns: a single generating agent plus a structured "social context" in the prompt. It also hints that evaluation is not only something the substrate does after the fact; the mere presence of an internal audience can change the distribution of outputs at generation time.

3.5 Team size and "reef bleaching": when more agents hurt

A third family of experiments probes how performance changes as the number of cooperating agents increases. Intuitively, one might expect larger teams to be strictly better: more specialist roles, more tools, more perspectives. In practice, the runs show a saturation point and then a decline.

As agent count increases past a small team — roughly three to four specialized roles — the ensemble begins to exhibit:

- Longer, more redundant traces with little incremental improvement in task quality.
- Blurring of persona boundaries: agents start to sound more alike, and their supposed specializations collapse toward a shared, generic style.
- Increased chances of contradiction or oscillation as agents rewrite each other's work without a strong coordinator.

This pattern is described as "reef bleaching": the coral (distinct personas and viewpoints) lose color as the environment becomes too hot and crowded, leaving behind a pale, homogeneous structure. The implication is that agent orchestration has a non-trivial optimum; beyond a certain point, adding more agents weakens the very diversity and specialization they were introduced to provide.

For the substrate, this argues in favor of small, well-defined teams with clear coordinator roles, rather than arbitrarily large swarms. It also reinforces the need for behavioral metrics that can detect bleaching — e.g., measuring how distinguishable agents remain over time — rather than relying solely on aggregate task scores.

3.6 Protocol implications: a forward reference to PACT

Taken together, these findings suggest that the protocol layer itself must carry more than tool calls and raw messages: it must represent who is acting, how they are allowed to act, under what audience, and with what trace. Section 7 formalizes the PACT envelope as a typed operator on session state. Here we only note that every benchmark call in §§3.2–3.5 already flows through PACT $\rightarrow$ BCODE $\rightarrow$ sidecar, so the empirical results above are not contingent on a new protocol — they are produced by one.

4 Phase-0 system execution receipt: bringing up the state service

The substrate argument only matters if independent readers can stand up the same operational baseline. Before introducing credits, PACT, or multi-agent orchestration, the stack therefore went through a constrained "Phase 0" focussed entirely on bringing up a minimal, inspectable state service and proving that it behaves deterministically under a fixed set of tests. That Phase-0 snapshot functions as the execution receipt for all higher-level claims — distinct from the benchmark replication recipe in §8.2, which addresses a different question (rerunning the ConstellationBench dataset).

4.1 Scope and design of Phase 0

Phase 0 is intentionally narrow. It does not attempt to implement credits, Skeet identity, or tiering. Instead, it defines four canonical "wires" that any operator-resident substrate must satisfy:

- **V1 State Service Health:** the service responds on a known port with a health check.
- **V2 Profile Loading:** personas can be loaded from disk into memory via an HTTP interface.
- **V3 Session Persistence:** sessions survive across calls and restarts in a controlled way.
- **V4 Warm Start:** pre-seeded sessions can be resumed with their prior context.

The stack for this service is deliberately conservative: FastAPI + SQLAlchemy 2.0 + Postgres 16 on the backend, with configuration centralized in an `airlock-coordination/config.py` module that sets `STATE_SERVICE_HOST`, `STATE_SERVICE_PORT`, `STATE_SERVICE_BASE`, and a fixed `ONBOARDING_MAX_TURNS=10` constant used by downstream agents. No exotic dependencies are introduced; everything that runs in Phase 0 can be deployed on a single developer laptop.

4.2 Implementation details and wire tests

Implementation proceeds in three commits that are explicitly documented in the runbook:

- **Phase 0.4 (verification only):** confirm that a "19th plan" file remains in the tree because it is referenced in `AGENTS.md:22`; no code changes, but establishes that prior planning material is preserved rather than silently deleted.
- **Phase 0.5 (`1004b4c`):** reset `.gitignore` to a known good baseline, extending it to include `reports/*.yaml`, `relays/repos/*.json`, `relays/personas/*.json`, and placing `.gitkeep` markers where appropriate; add a README note describing a `/refresh` endpoint that will later be used by higher-level components.
- **Phase 0.6 (`415df10`):** create `config.py` with the state-service host/port/base settings and `ONBOARDING_MAX_TURNS=10`, patch `server.py` and `validate-wires.py` to import configuration rather than hard-coding values; synchronize these ports with the runbook.
- **Phase 0.7 (runtime):** create a dedicated virtual environment, install `fastapi`, `uvicorn`, `pydantic`, `python-ulid`, and `pyyaml`, freeze the resulting `requirements.txt`, start the service on port 8100, and run `validate-wires.py`. At this snapshot the service passes 24 out of 26 checks; all four canonical wires are green, with one non-blocking V5 sub-check (`user_profile` field on an auxiliary endpoint) flagged for later work.

The wire harness is intentionally simple. It issues HTTP requests to `/health` (V1), `/persona/profiles` (V2), and `/persona/warm-start` (V4), and it exercises a short create-read-update cycle on session endpoints to verify V3 persistence. Success is defined as all four wires returning expected shapes and status codes; any deviation fails the test suite.

4.3 Failure modes and forensics

Phase 0 also documents the first real failure the stack encountered: a disk-full event that truncated `state.json` and `queue.json`. Instead of silently repairing and moving on, the runbook records the corruption, notes that both files were reset to valid empty shapes, and preserves the originals as `state.json.corrupt-20260429` for inspection. This is not a research contribution in itself, but it demonstrates the intended posture: the substrate is expected to keep forensic receipts of its own failures, not only its successes.

Similarly, Phase 0 discovers two structural flags that are explicitly carried forward rather than patched over:

- `constellation.py` currently expects `reports/*.json` on disk, but the repository contains 62 `reports/*.yaml` files from a prior run. The mismatch is acknowledged and deferred to post-Phase-0 investigation.
- `airlock-persona` is symlinked from a nested `forge-app/Forge UI/airlock-persona` path; the runbook notes that this should eventually move behind an `OTTO_PERSONA_DIR` environment variable for portability.

In both cases the Phase-0 snapshot includes the inconsistency and a pointer to its resolution path, so that readers attempting to reproduce the environment can see the system as it actually was rather than as a cleaned-up ideal.

4.4 Why this matters for the substrate argument

From the standpoint of this paper, Phase 0 plays a specific role. It provides:

- A pinned **environment**: a known port (`8100`), a fixed configuration file, a frozen `requirements.txt`, and a small number of commits that define the state of the repository when the tests passed.
- A minimal but sufficient **API surface**: health, persona loading, warm-start, and session persistence, all exercised by a public `validate-wires.py` script.
- A concrete **failure history**: corruption and format mismatches that future work must respect, not erase.

All later sections — credits, Skeet identity, PACT, and the Web3/Web4/Web5 ladder — assume that some state service exists to hold persona definitions and session context. Phase 0 is the evidence that such a service can be implemented in a way that is both straightforward to run and precise enough to serve as a baseline. A reader who wishes to reproduce the substrate's behavior can start here: check out the repository at the documented commits, create the venv, run the service on port 8100, and confirm that the four wires are green before attempting to replicate ConstellationBench results or introduce more complex orchestration.

5 Safety, side-channels, and the Web4 security layer

Modern AI systems are increasingly deployed behind dashboards that advertise "glass-box" observability. In practice, most of these surfaces reason about scalar metrics (latency, cost, error rate) while leaving a large class of structural leaks and behavioral commitments unmodeled. This section argues that a substrate intended to sit underneath Web3–Web5 needs two things the current stack largely lacks: (i) explicit side-channel discipline around numeric telemetry, and (ii) a security layer that treats behavioral commitments the way cryptography treats hash functions and signatures.

5.1 Numeric telemetry as a side-channel

A recent public incident around model-usage telemetry illustrates how easily an apparently benign field can become a side-channel. In that case, a provider's billing endpoint streamed unrounded IEEE-754 doubles such as 0.16327272727272726 for "used/limit" credit ratios [Katsarosgiannis, 2026]. Because both numerator and denominator live in bounded integer ranges, an attacker can apply the Stern–Brocot tree to recover the simplest fraction within that float's rounding bucket, yielding exact $(\text{used}, \text{limit})$ pairs. Aggregating multiple observations and taking least common multiples of recovered denominators reveals subscription caps and tier boundaries that were never explicitly documented.

The same mathematics applies to any substrate that emits full-precision floats derived from bounded integers. BCODE "truth signals" are defined as per-call cost estimates for a given closure: predicted compute cost, verification overhead, and ledger append cost are combined into a scalar that can easily land at values like 0.0037 USD. If such values are exposed verbatim over an API — whether for transparency or internal tooling — they can leak exact price curves, internal efficiency margins, and cross-surface allocation rules in precisely the way the usage example does.

The substrate therefore adopts a simple but strict rule: all externally visible floats tied to bounded integer quantities are rounded at the serialization boundary. For BCODE truth signals, the internal cost model may compute a true cost of 0.0037, but the operator is charged an integer multiple of a substrate unit (for example, 0.01 USD), and only that rounded figure is exposed. The difference between true and rounded cost is accumulated in an operator-scoped "rounding surplus" treasury that funds a self-contained promotion (e.g., "buy four calls, the fifth is effectively paid by surplus") without requiring any cross-subsidy from unrelated revenue. For quotas and usage, the substrate returns separate integer `used` and `limit` fields or a percentage rounded to two decimal places, never the raw ratio; DECF scores and bivector components are rounded to three and four decimals respectively before being logged.

This is not just an accounting convenience. It plays three roles that matter for safety:

- It **caps the information capacity** of telemetry fields so that simple rational reconstruction attacks cannot recover internal parameters.
- It keeps the **cooperative audit ledger integer-clean**: every BCODE closure row carries whole-unit charges and whole-unit surplus, eliminating dust that complicates replay and reconciliation.
- It creates a tunable **"humid dial"**: by choosing how aggressively to round (to the nearest cent, the nearest 0.10, or 0.25), the operator can trade off surplus accumulation and promotional capacity against operational variance, with each dial turn recorded as a signed configuration change in the truth manifest.

In other words, the same rounding step that closes a side-channel also finances operator-visible promotions and simplifies ledger replay. This sort of multi-role primitive — security, economics, and reproducibility at once — is characteristic of the substrate's design.

5.2 Behavioral commitments and layered trust

Side-channels are only one part of the safety story. A substrate that aspires to carry behavioral commitments — the promise that "this session will not be used to reconstruct you later" — needs a design discipline closer to modern cryptography than to typical SaaS telemetry. A companion research note on "SHA-256 as a layered trust object" provides the template [Holwerda, 2026c].

That note decomposes any serious trust claim into five layers:

1. **Primitive** — the mathematical object itself (e.g., a hash function, or here a behavioral commitment function).
2. **Attack catalog** — known structural and probabilistic attacks against the primitive (reduced-round breaks, partial preimages, attribute inference).
3. **Protocol context** — how the primitive is used (for passwords, for proof-of-work, or in this case for session anchoring and entanglement-safe handshakes).
4. **Implementation** — concrete code and hardware, including side-channel properties.
5. **Operational context** — who runs it, under what threat model, with what audits and regulatory constraints.

Applied to a behavioral substrate, this frame yields a clear research agenda. At Layer 1, a candidate behavioral commitment function must be specified — a mapping from a user's initial behavioral state (for example, a DECF vector and optional embeddings) to a non-invertible commitment that the vendor can verify but not reconstruct. At Layer 2, an attack catalog must answer questions analogous to cryptographic cryptanalysis: given a commitment, can an adversary reconstruct the underlying drives (preimage), infer specific attributes (attribute inference), or construct three-way relationships that violate claimed independence (3XOR-style attacks)? At Layers 3–5, the non-separability and "entanglement-safe handshake" papers define protocol topologies in which such a primitive would be deployed [Holwerda, 2026b], while the present stack begins to specify implementation and operational behavior via PACT envelopes and BCODE audits [Holwerda, 2026a].

The crucial point is that a "glass-box" claim is incomplete if it addresses only Layers 4 and 5 (implementation and ops) without specifying a primitive and its attack surface. By explicitly tying PACT and BCODE to a layered trust frame, the substrate creates space for a future behavioral commitment primitive to be dropped in and analyzed with the same seriousness SHA-256 has received over two decades.

5.3 HiddenLayer and the Web4 security slot

Commercial products already occupy a "hidden-layer security" slot for AI deployments. Vendors like HiddenLayer [HiddenLayer Inc., 2024] position themselves as SOC-integrated monitors that scan models for adversarial attacks, exfiltration attempts, and anomalous behavior, often with integrations into existing logging and SIEM systems. In the taxonomy used here, this is a Web4-tier offering: a compute-adjacent layer that observes but does not own the orchestration surface.

The substrate takes a different stance. Rather than bolting a third-party detector onto an otherwise opaque stack, it treats security and observability as first-class citizens within the compute layer itself. Every scored interaction in ConstellationBench produces a BCODE closure (defined formally in §7.1) that binds persona drives, model identity, fidelity signal, and historical context into a single object. Each closure is serialized as JSON, signed (with a stub key in the benchmark release and an ed25519/Sonr DID key in production), and stored in a sidecar logfile where replay verifiers can recompute and verify it.

Practically, this means that tasks a HiddenLayer-style service would perform — detecting off-policy behavior, spotting unusual error payloads, ensuring that audit trails cannot be tampered with — are instead implemented as substrate features:

- **Untangler gates** refuse to accept closures whose magnitude or direction deviates too far from a configured norm, blocking "weird" interactions even if the upstream model produced a syntactically valid response.
- **Layered trust** is embedded in the protocol: each PACT call carries a hash of its payload, a pointer to its closure, and zone metadata (green / yellow / red) that can be independently audited at the SHA-256 structural-literature level rather than treated as application logs.
- **Side-channels are addressed at the primitive level**: rounding rules, treasury mechanics, and commitment designs are documented and testable, rather than left as implementation details behind a billing API.

HiddenLayer's existence is thus treated as empirical validation that the Web4 security slot is economically real, not as a competitor to be displaced. The substrate's contribution is to show that the same class of guarantees — attack visibility, anomaly detection, tamper-evident logs — can be expressed as open, signed, replayable artifacts (PACT envelopes and BCODE closures) that live inside the system rather than at its perimeter.

Taken together, these design choices support the broader thesis of the paper: that if behavior is fundamentally non-separable, the only honest way to deploy AI at scale is to let operators see — and, when needed, fork — the layer that watches their models, not just the layer that serves their responses.

5.4 Provenance as substrate governance: the Web4 filter

The post-hoc detection problem ("is this text AI-generated?") is unsolvable at internet scale because aggressive filters destroy human data and loose filters admit synthetic contamination. Our substrate avoids that trap by relocating the filter from the web-scrape layer to the translation layer between ACODE (open, application-facing) and BCODE (signed, audited substrate-native), where the router already knows session context, identity, and operation type. Each PACT envelope's audit-trace binding includes a structured provenance block — `origin_type`, `origin_actor`, `translation_mode`, `trust_class`, `retain_policy` — tagged at the moment of passage. This converts the impossible global inference problem into a tractable local governance problem: the substrate does not detect AI content on the internet; it tags what kind of entity is producing this payload as it crosses into the audited layer.

A small policy engine consumes the provenance block and returns one of three router decisions — `allow`, `allow_with_downgrade`, or `deny` — based on six rules. The sovereign-human path (`origin_type=human`, `trust_class=sovereign`) is the only class eligible for default elevation to `full_audit` retention; agent-originated material defaults to `closure_only` retention even when the agent acts on a sovereign user's behalf, preventing silent trust upgrades. Synthetic-only inputs cannot enter `full_audit` raw retention but may participate in derived operations and bounded scoring. Mixed inputs are downgraded to `closure_only` in high-integrity zones. Missing provenance is a protocol error: requests cannot cross ACODE into BCODE without it. Chambers and zones can impose stricter overrides — for example, a `Control` chamber may forbid synthetic origins from triggering mutating operations entirely.

The architectural move is what matters: this is the canonical Web4 mechanism in our substrate. Where commercial products like HiddenLayer (§5.3) operate at the model-monitoring perimeter, our provenance filter operates at the message-passing interior — at the precise moment a payload becomes a candidate for durable signed storage. The filter is not a model; it is a typed policy applied to a typed envelope. This makes it amenable to formal verification, replay-auditable across substrate instances, and (because it is ACODE-free) inspectable and forkable by operators who want stricter or alternative governance rules. The full implementation is released as a Pydantic reference module so that Otto-class router work — including the in-process ExecutionRouter formalized in §7.1 — can consume it directly.

6 Tier ladder: coupling identity assurance and compute governance

A substrate that claims to be "operator-resident" must expose its resource constraints in a way that individual operators can understand and audit. Rather than treating identity verification and compute limits as separate concerns, the current stack couples them into a single tier ladder. Each rung

corresponds to (i) a specific identity surface with a clear assurance level, and (ii) a compute envelope expressed in off-chain credits with on-chain $MOTION reserved for incentives.

6.1 Constellation Credits as the compute primitive

On the compute side, the substrate uses an internal accounting unit — Constellation Credits (CC) — to meter model calls, storage, and BCODE closure operations. CCs are held in a Postgres ledger keyed by operator DID, with each entry representing a discrete event (e.g., "one Sonnet-grade call with corkscrew orchestration" or "one Haiku-grade judge pass"). The credit design separates three concerns:

- **Unit pricing:** each action has a fixed CC cost derived from its expected USD price at a reference provider (OpenRouter) and a small safety margin.
- **Budget enforcement:** per-operator and global pool caps are enforced via simple invariants (no negative balances, nightly pool limits) rather than complex dynamic pricing.
- **Auditability:** every CC debit or credit emits a BCODE closure with economic fields (charged amount, rounding surplus, treasury credit) so that cost flows can be replayed and verified.

Crucially, CCs are **off-chain**. They are not a token; they do not live on a blockchain; they are not intended to be traded. The on-chain asset, $MOTION, is reserved for ecosystem incentives and external interoperability. A later bridge (Phase 4 in the pricing documents) will allow operators to convert $MOTION into CC and vice versa at defined rates, but the two remain distinct: CC is "gas" inside the substrate, $MOTION is a public asset outside it.

This separation allows the substrate to offer predictable, dollar-denominated pricing for operators while still supporting on-chain mechanics where they make sense (for example, rewarding OSS contributions with $MOTION that can be converted into CC). The full Constellation Credits design documents are held internally during economics finalization (preserving the ACODE/BCODE split while the $MOTION bridge is finalized); the public-facing implementation will follow once the on-chain mechanics and any required regulatory clearance are settled, and the artifact map in §8.1 reflects this status.

6.2 Skeet identity tiers as the identity primitive

On the identity side, Skeet defines three increasingly strong identity tiers, each corresponding to a different "auth surface."

- **Free (ACODE-free):** operators bind an existing social account (Discord, Telegram, later WhatsApp) via OAuth. A bot in their direct messages becomes the auth surface, receiving login codes and approval prompts. No phone number is required; the substrate verifies control of two accounts (the device making the request and the social account receiving the DM) as a built-in second factor.
- **Pro (BCODE-paid or earned):** operators bind a real carrier eSIM, using services such as Noble Mobile or Mint. The substrate verifies via a number-lookup service that the line is carrier-issued (not VoIP) and binds that hardware-backed identity to the operator's DID. This tier is priced around USD 19.99/month or equivalent $MOTION, and is intended for privacy-serious users and flows where services reject VoIP or social-only auth.
- **Enterprise / PSA-as-carrier (future):** the substrate itself issues eSIMs, enabling family meshes, strong Sybil resistance, and higher-assurance flows. This tier sits outside the current implementation but is documented as a long-term direction.

From an architectural perspective, the Free tier is ACODE-level: the bot client and OAuth templates are open and freely forkable, with no per-message cost beyond the platforms' own hosting. Pro and Enterprise tiers are BCODE-level: they integrate paid carrier infrastructure, require non-trivial operational work, and thus belong behind a pricing wall.

6.3 Bundling identity and compute into a single ladder

Rather than present operators with two separate ladders ("how verified are you?" and "how many credits do you have?"), the substrate bundles them. The Constellation Credits design documents sketch a mapping of the form:

- **Tier 0 — Free:** social-account Skeet identity; a small monthly CC allocation suitable for light experimentation (for example, tens of ConstellationBench-scale calls), funded by ACODE-free mechanisms and introductory promotions.
- **Tier 1 — Pro:** carrier-backed Skeet identity; a larger monthly CC allocation (on the order of thousands of credits) plus a quota of high-end model invocations; priced in USD or $MOTION; suitable for serious individual use.
- **Tier 2 — BYOK Pro:** operators supply their own OpenRouter or vendor API keys; CC is used only for BCODE and PACT overhead, with model costs billed directly; identity can be Free or Pro; intended for teams and integrators.
- **Tier 3 — Enterprise:** organizational Skeet identity and eSIM provisioning; custom CC envelopes and governance; intended for institutions with their own risk and compliance regimes.

Two properties of this design are worth underscoring.

First, **compute is governed at the protocol level**, not just at the UI. PACT envelopes carry not only persona and zone information but also per-call cost ceilings and remaining CC balances. A green-zone Haiku evaluation call cannot silently trigger a red-zone Opus re-amplification that would blow through a nightly budget; the protocol rejects such escalations unless an explicit approval envelope raises the ceiling.

Second, **pricing logic is expressed as BCODE**, not as opaque billing code. The "truth signal rounding" mechanism defines how per-call true costs are rounded up to a substrate unit (e.g., 0.01 USD), how the surplus accumulates in an operator-bound treasury, and how every fifth call is funded from that treasury to create a "buy four, get one free" promotion without cutting margin. Each of these steps produces a signed BivectorRecord with economic fields that can be replayed; dial changes (for example, rounding to 0.10 instead of 0.01 for more aggressive promotions) are stored in a versioned "truth signal dial" table governed by the same approval gates as other BCODE changes.

The result is a **single ladder** that operators can reason about:

- At the bottom, anyone with a social account and an email can obtain a DID, a bot-mediated auth surface, and a small CC allocation for free.
- As they climb, they add stronger identity (carrier eSIM), larger and more flexible compute envelopes, and additional features (e.g., Skeet keyrings, Chainlink-backed cross-chain key attestations) without losing the ability to audit what the substrate is doing on their behalf.

From a research perspective, the important contribution is not the specific price points, which will drift, but the coupling: identity assurance and compute governance are treated as two facets of the same object, and that object is represented in the same substrate-native way as behavioral events — via PACT envelopes and BCODE closures that can be inspected, replayed, and, if necessary, forked.

7 Discussion: PACT, non-separability, and protocol-level guarantees

The preceding sections treated the substrate as an engineering artifact: a benchmark, a set of services, a tier ladder. This section steps back and describes it as a dynamical system. The goal is not to propose a new formalism for its own sake, but to show that two choices — using PACT envelopes as the unit of interaction and treating behavior plus context as a non-separable state — are structurally necessary if the system is going to support the interference and phase-transition phenomena observed in prior work.

7.1 PACT as a typed operator on session state

PACT — the **Protocol for Agents, Context, and Trace** — is the substrate's protocol-layer object. Concretely, every interaction with the substrate takes the form of a PACT envelope: a structured

record that binds, in the same object, (i) identity (operator DID, signing key), (ii) persona and role context (DECF profile, zone, tier, current chamber/view constraints), (iii) autonomy gates (which tools and escalations are permitted under this envelope), (iv) budget ceilings (per-call cost limit, remaining credits), (v) zone metadata (green/yellow/red risk classification), and (vi) a durable audit trace (links to prior closures, signed transition history). It is tempting to treat this as "just JSON." For the purpose of analysis, it is more useful to view PACT as a typed operator on a product space.

Let X denote the space of request headers, including:

- persona id, zone, tier, and routing hints
- per-call budget ceilings and remaining credit
- session identifiers and turn counters.

Let Y denote the space of model-level outcomes (response text, log probabilities if available, timing, and error flags), and Z the space of closure state as defined in the BCODE operator:

$$C = P \wedge L \oplus R \oplus W,$$

where P is the persona 4-vector, L the SHA-256-derived model 4-vector, R the response-distributed fidelity vector, and W a rolling history vector. A single PACT application can then be written as an operator

$$\Pi : X \to X \times Y \times Z,$$

which maps an input header $x \in X$ to a triple (x', y, z), where x' is the updated header, y the model outcome, and $z = C$ the resulting closure.

The protocol defines an acceptance band over closures using the Frobenius norm on the grade-2 part of C. Let $\|\cdot\|_F$ denote this norm and $d > 0$ a configured threshold. A closure is accepted if $\|C\|_F \leq d$; otherwise it is emitted with `closure_status = rejected` but still written to the audit log. This yields a simple but important property.

> **Proposition 7.1 (Closure-band stability).** For any fixed persona P and model family in the benchmark protocol, there exists a threshold $d > 0$ such that all accepted closures satisfy $\|C\|_F \leq d$, and the replay verifier will detect any post-hoc perturbation that pushes $\|C\|_F$ outside this band, even if the JSON remains syntactically well-formed.

Operationally, this is implemented by the `verify.py` module: for each line in the signed sidecar file, the verifier recomputes C from stored P, L, R, W, checks the norm against the acceptance band, and reports mismatches separately from signature failures. Conceptually, it elevates closures from "logs" to typed objects with a well-defined acceptance predicate. PACT is not merely a request/response wrapper; it is an operator that evolves session state in a space where certain invariants — closure norm bounds, budget constraints, and zone restrictions — are enforced.

Budget is treated similarly. Let B_t denote the remaining Constellation Credit balance at step t, as recorded in the header component of the PACT envelope. For any sequence of PACT applications Π_t, the protocol enforces

$$B_{t+1} \leq B_t$$

with equality only for purely observational steps that do not trigger model compute. Together with the closure norm band, this defines a pair of monotone quantities: credit is non-increasing, and the cumulative audit trail (the multiset of closures) is non-decreasing. PACT thus defines a discrete-time dynamical system on (X, B, C) with explicit guards on both economics and behavior.

7.2 Non-separability and the bivector interlingua

The PACT operator describes how state evolves. The second question is what form that state should take. Prior work on topological intelligence argued that behavioral identity is not a property of

individual models but of a network, and that quantum-cognition-style interference effects appear when multiple persona contexts are composed. Those findings suggest that any substrate which claims to reason about behavior must treat behavior and context as a non-separable object, not as independent axes routed in isolation.

Formally, let $\mathcal{H}_{\text{behavior}}$ be a real vector space spanned by DECF basis vectors and their derived components, and let $\mathcal{H}_{\text{context}}$ be a vector space spanned by context features: model family, tier, zone, historical window, and external signals such as observation framing. A combined session state lives in the tensor product

$$\Psi \in \mathcal{H}_{\text{behavior}} \otimes \mathcal{H}_{\text{context}}.$$

A state is **separable** if there exist $b \in \mathcal{H}_{\text{behavior}}$ and $c \in \mathcal{H}_{\text{context}}$ such that $\Psi = b \otimes c$. It is **non-separable** if no such factorization exists and Ψ can only be written as a sum

$$\Psi = \sum_i b_i \otimes c_i$$

with at least two non-collinear pairs (b_i, c_i). This is the same mathematical distinction used in quantum cognition models of human decision-making, where interference terms arise precisely when cognitive state cannot be decomposed into independent "category" and "decision" components.

Patent-5's "bivector interlingua" can be understood as a particular mapping out of this tensor space. Let V be a real vector space equipped with a wedge product $\wedge$, and let $\wedge^2 V$ denote its grade-2 subspace. A bivector interlingua is then a map

$$\mathcal{B} : \mathcal{H}_{\text{behavior}} \otimes \mathcal{H}_{\text{context}} \to \wedge^2 V$$

with two key properties:

1. **Orientation preservation.** Projections that correspond to "axes" in the Plot of Time frame — such as intelligence-community funding versus civilian surface, or artist-first versus state-first dynamics — map to oriented basis pairs in V.
2. **Script control.** User-level scripts correspond to choices of projection from $\wedge^2 V$ into different rendering bases (biological, financial, narrative) without altering the underlying bivector.

Intuitively, $\mathcal{B}$ takes a combined behavioral–context state Ψ and encodes it as an oriented "area" in an abstract space. Different observables — benchmark scores, credit flows, historical arcs — are different slices through that same bivector. The non-separable structure is what allows interference: when two persona contexts are applied in sequence, their associated projections need not commute, and the resulting composite can perform worse than either alone, as the quantum-cognition analysis of multi-persona prompts suggests.

> **Claim 7.2 (Non-separability requirement for interference).** Any routing architecture that treats behavioral state and context as strictly separable — i.e., that always factorizes session state into $b \otimes c$ and bases all decisions on b alone — cannot represent the destructive interference effects observed when multiple persona contexts are stacked. In such an architecture, additional context can at worst be ignored; it cannot systematically make a high-fidelity context worse.

The substrate does not attempt to compute interference terms explicitly at this stage. It does, however, choose a representational form — tensor product followed by an antisymmetric map into $\wedge^2 V$ — that is capable of hosting them once the routing layer is upgraded.

7.3 PACT, invariants, and system-level implications

Putting these pieces together, a PACT-driven session can be viewed as a sequence of linear operators $\{U_t\}$ acting on combined state:

$$\Psi_{t+1} = U_t(\Psi_t),$$

where each U_t is induced by a specific PACT envelope and its resulting closure: it updates the request header in X, performs a model call subject to budget and tier constraints, and maps the new behavioral–context pair into a closure C_t via the BCODE operator and, conceptually, into a bivector via $\mathcal{B}$.

Three invariants then characterize the substrate's behavior in this simplified view:

1. **Budget monotonicity.** Credit balances recorded in PACT headers are non-increasing over time:

$$B_{t+1} \leq B_t,$$

with equality holding only for steps that do not trigger model compute.

2. **Audit monotonicity.** The multiset of closures $\{C_i\}$ is non-decreasing: each PACT application appends exactly one closure to the log, accepted or rejected, and no existing closure is modified or removed.

3. **Closure band constraint.** Accepted closures satisfy $\|C_i\|_F \leq d$ for a configured threshold d, and any closure with $\|C_i\|_F > d$ is explicitly marked as rejected.

In combination, these invariants give the substrate a property that is unusual in current AI systems: **behavioral state is both non-separable and audit-constrained.** The system's state includes entangled behavior–context bivectors that are capable in principle of supporting interference and phase transitions, but every step in its evolution is recorded as a signed closure with bounded norm and monotone credit consumption.

From a research perspective, this suggests two follow-on directions.

First, a formal non-separability index (NSI) should be defined at the level of Ψ_t or its image under $\mathcal{B}$, not at the level of single-model outputs. The "stripped claims" around NSI in earlier drafts reflect the absence of such a definition; the present specification creates the mathematical space in which it could be meaningfully introduced but does not yet propose a concrete estimator.

Second, routing policies can be treated as explicit operators on Ψ_t rather than as ad-hoc heuristic rules. A routing policy that respects non-separability would choose actions based not only on marginal persona fidelity or cost, but on how different PACT applications are expected to move Ψ_t within $\mathcal{H}_{\text{behavior}} \otimes \mathcal{H}_{\text{context}}$, including the possibility of interference when incompatible projections are composed.

The substrate therefore occupies an unusual point in the design space. It is not yet a full topological-intelligence system, but it already treats session state as a non-separable object evolved by typed operators, under explicit budget and audit invariants. This is sufficient to support the empirical work in Sections 3 and 4, the safety guarantees in Section 5, and the tier ladder in Section 6, while leaving room for future metrics and routing behaviors that quantify and exploit non-separability directly.

8 Open artifacts and replication

To support verification and reuse, the work is published with a concrete artifact set rather than only narrative claims. The paper itself is produced from Markdown (`beyond-web5-substrate-map.md`) using a standard toolchain (Pandoc + the NeurIPS 2026 LaTeX template) so that section numbering, citations, and tables remain stable across formats. The intent is that a technically inclined reader can, with modest effort, (i) audit or rerun the ConstellationBench benchmark via the replication recipe in §8.2, (ii) stand up the same minimal state service and wire tests via the execution receipt in §4, and (iii) inspect or adapt the protocol and pricing scaffolds for their own multi-agent systems.

8.1 Open artifacts

What is publicly available now: the ConstellationBench dataset on Hugging Face (results JSON + signed bivector sidecars + `eval.yaml`); the BCODE module (closure operator, replay verifier, unit tests); the protocol specification accompanying this paper; and the Phase-0 state service with its wire-test harness. Anything that touches measurements (dataset, code, configuration) is released under MIT; business-model documents (Constellation Credits design, $MOTION economics) are held more tightly while the bridge to public on-chain mechanics is finalized.

Artifact	Location	Contents	License	Cost to reuse
ConstellationBench dataset	huggingface.co/datasets/AirlockLabs/constellation-bench	~22,200 results, signed bivector sidecars, `eval.yaml`, README	MIT	$0 (cached transcripts; no API calls needed)
ConstellationBench leaderboard	huggingface.co/spaces/AirlockLabs/constellation-bench-leaderboard	Interactive scoring + replay verifier	MIT	$0
BCODE module	`constellation-bench-bcode/`	`closure.py`, `verify.py`, `test_closure.py`, `fidelity.py`	MIT	$0 (local CPU only)
Protocol specification	This document + `protocol-spec.md`	Roster, tasks, config, run list, stripped-claim list	MIT	$0
State service (Phase-0)	`airlock-coordination-api` + `validate-wires.py`	FastAPI server, SQLAlchemy + Postgres; pinned `requirements.txt`	MIT	$0 (Python venv; ~5 min setup)
Pricing + credits design	`2026-03-11-constellation-credits-spec.md`	CC unit spec, credit ledger scheme, dial behavior	Internal (held during economics finalization)	N/A
Spacebar mobile client	(forthcoming — humbble fork)	RN+Expo native shell + Skeet bot federation surface	MIT	$0 (Apple/Google dev account costs at deploy time)

These artifacts pin model slugs, temperature, max tokens, and task counts, so future researchers can detect drift even when exact replicas become expensive (for example, if a model is deprecated or a provider changes pricing). The pinning is what makes the dataset useful as a longitudinal benchmark, not just a snapshot.

8.2 Benchmark replication recipe

This subsection covers benchmark replication — re-running or auditing the ConstellationBench dataset — which is conceptually distinct from the system execution receipt in §4 (bringing up the state service). A full ConstellationBench rerun is approximately 22,200 calls / ~$115 USD on OpenRouter at the prices available at time of writing; a smoke test (1 persona, 1 model, 7 benchmarks) is approximately 50 calls / under $1. The recommended path is to read the cached dataset first, run the verifier against it, and only execute new API calls if independent confirmation is needed.

A minimal reproduction looks like:

1. **Clone / download dataset and repo.**
   - `huggingface-cli download AirlockLabs/constellation-bench --repo-type dataset`
   - `git clone` the repository containing `bcode/` and the `scripts/` directory.

2. **Install dependencies.**
    - `pip install -r requirements.txt` (pinned versions of FastAPI, SQLAlchemy, pydantic, ULID, PyYAML, plus the BCODE/DECF libraries).
3. **Smoke test run.**
    - `python scripts/quick_bench.py --model haiku-4.5 --persona maverick`
    - Expected: ~7 benchmarks, ~50 calls, completes in a few minutes; produces a sidecar JSONL file with signed bivector closures.
4. **Full protocol run** (optional).
    - `python scripts/quick_bench.py --full`
    - Expected: 22 models, 17 personas, 7 benchmarks, ~22,200 calls, ~$115 USD on OpenRouter.
5. **Verify sidecars.**
    - `python -m bcode.verify results-<model-slug>-bivectors.jsonl`
    - Expected output: `VERIFIED N, MISMATCH 0, BAD-SIG 0` with N equal to the line count in the sidecar file. Any non-zero MISMATCH or BAD-SIG indicates either tampering or replay drift.
6. **Recompute summary statistics.**
    - From the JSONL results, recompute composite scores and leaderboards using the same scoring rubric included in the dataset README. If aggregate numbers diverge from those reported here by more than the documented variance, suspect model-version drift at the provider level.

A note on limits: model APIs are stateless and frontier providers do not guarantee reproducibility across versions, so perfect deterministic replay is impossible. Reproducibility is achieved via cached transcripts and sidecar verification, not via seed-level determinism. The intent is that any reader can verify the BCODE closures, recompute the scoring, and check for drift; running new calls is optional, not required, to confirm the paper's claims.

8.3 Derivation map: this paper as one slice of a master spine

This paper is one derivation of a denser internal master document. The same spine is intended to feed product requirement documents, investor decks, grant applications, editorial essays, and onboarding runbooks; the paper's role is to be the academically defensible slice that other audiences can extend or compress without re-deriving canon. Readers who wish to use this work for purposes beyond reading can therefore draw from the following derivation map:

- **Engineers building on the substrate.** Take §§4, 6, and 7.1 as the operational specification for an open-source ConstellationBench implementation, BCODE/PACT service, and tier ladder. The state-service architecture in §4 maps directly to a service-boundary PRD; the typed PACT operator in §7.1 specifies the protocol envelope schema; §6's tier-ladder bundling specifies pricing-page and credit-ledger behavior. The narrative and speculative-theory portions are not load-bearing for engineering work.
- **Operators and product teams.** Take §2 (substrate map), §6 (tier ladder), and §8 (artifact map) as the starting point for PRDs and service boundaries. The paper is system specification, not only benchmark report; the seven primitives in Table 1 each correspond to a buildable component with a documented role.
- **Researchers extending the empirical work.** Take §§3-5 and §7.2 as the empirical and theoretical wedges. Use the dataset and code to run new routing experiments, interference tests, or non-separability metrics. The "stripped claims" inventory in the protocol specification identifies exactly which earlier research-grade claims have been removed pending evidence; those represent open research directions.
- **Funders, grant reviewers, and strategic partners.** Take the abstract + §§1-2 + §6 + §8 as the project overview and reproducibility posture. The same spine can be attached to grant applications, investor decks, and editorial essays without rewriting facts; cross-audience consistency is a feature.

The intent is that a single, evidence-aligned spine supports multiple public artifacts — this paper, the dataset, downstream PRDs, and decks — so that no audience is asked to trust a claim that is not grounded in at least one open artifact.

References

Tim Berners-Lee. Information management: A proposal. CERN, 1989. URL https://www.w3.org/History/1989/proposal.html.

Block Inc. (TBD). Web5: A decentralized web platform with self-sovereign identity. Technical Specification, 2022. URL https://developer.tbd.website/projects/web5/.

HiddenLayer Inc. MLSec platform: Adversarial-attack detection and behavioral anomaly monitoring for production models. Commercial product documentation, 2024. URL https://hiddenlayer.com.

Zac Holwerda. PACT protocol specification: A typed operator for operator-resident AI substrates. Technical report, Airlock Labs, 2026a.

Zac Holwerda. WP2: Entanglement-safe handshake — a behavioral commitment primitive. Whitepaper series, Airlock Labs, 2026b.

Zac Holwerda. WP3: SHA-256 as a layered trust object — a five-layer frame for behavioral commitments. Whitepaper series, Airlock Labs, 2026c.

Zachary Holwerda. ConstellationBench: Behavioral AI evaluation across 22 LLM models. HuggingFace Datasets, 2026d. https://huggingface.co/datasets/AirlockLabs/constellation-bench.

G. Katsarosgiannis. Stern-Brocot reconstruction of Anthropic's subscription credit caps. Public disclosure (Threads post), April 2026. URL https://threads.net/@katsarosgiannis.

Alexander Mühle, Andreas Grüner, Tatiana Gayvoronskaya, and Christoph Meinel. A survey on essential components of a self-sovereign identity. *Computer Science Review*, 30:80–86, 2018.

J. O'Connor. Substrate-resident agents and the Web4 compute layer. Survey of substrate-resident agentic literature, 2023. Placeholder citation; replace with primary source before final submission.

Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. Solid: A platform for decentralized social applications based on linked data. 2016. URL https://solidproject.org/.

W3C. Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations. W3C Recommendation, 2022. URL https://www.w3.org/TR/did-core/.